

Lekce 5: Úvod do DOM: Vlastnosti, metody a globální objekt `window`

1. Osnova obsahu

A. Úvod do DOM (Document Object Model)

- **Co je DOM?**
 - Definice a účel.
 - Reprezentace webové stránky jako stromové struktury.
- **Přístup k prvkům DOM:**
 - `document.getElementById()` .
 - `document.querySelector()` a `document.querySelectorAll()` .
- **Manipulace s prvky DOM:**
 - Změna obsahu (`innerText` , `innerHTML`).
 - Úprava atributů (`setAttribute()` , `getAttribute()`).
 - Přidávání a odebrání tříd (`classList.add()` , `classList.remove()`).
- **Zpracování událostí:**
 - Přidávání posluchačů událostí (`addEventListener()`).
 - Běžné události (click, input, submit).

Co je DOM?

Definice:

Document Object Model (DOM) je programové rozhraní, které reprezentuje strukturu HTML nebo XML dokumentu jako strom objektů. Každý prvek, atribut a textový úsek se stává objektem, což vývojářům umožňuje programově manipulovat se strukturou, stylem a obsahem stránky.

Klíčové vlastnosti:

- **Stromová struktura:** Dokument je reprezentován jako hierarchický uzlový strom.
- **Skriptovací rozhraní:** JavaScript může pracovat s uzly DOM pro změnu zobrazení v prohlížeči.
- **Dynamické aktualizace:** Měnit prvky, atributy a obsah za běhu bez nutnosti reloadovat celou stránku.

Použití v testovací automatizaci:

- Výběr prvků pro ověření jejich přítomnosti, atributů nebo textu.
- Simulace uživatelských akcí (klikání, psaní) na prvcích DOM.

- Kontrola, zda se prvky objeví nebo zmizí po určitých akcích dle očekávání.

Zpracování událostí v DOM

Definice:

Zpracování událostí znamená detekci a reakci na uživatelské interakce nebo události řízené prohlížečem na webové stránce. Události mohou zahrnovat kliknutí, stisk klávesy, odeslání formuláře, pohyb myši, nebo jiné uživatelské akce.

Klíčové vlastnosti:

- **Posluchače událostí:** Funkce, které se spustí v reakci na konkrétní události.
- **Asynchronní chování:** Události probíhají asynchronně, vyvolané uživatelem nebo naplánovanými akcemi.
- **Kontrola nad uživatelskou interakcí:** Umožňuje vývojářům a testerům simulovat a ověřovat reakce aplikace na vstupy od uživatele.

Použití v testovací automatizaci:

- Testování interakcí v UI, kontrola, že kliknutí na tlačítko spustí správné chování.
- Ověření logiky odeslání formuláře nebo správné zpracování chyb.
- Potvrzení, že události stisknutí klávesy způsobí očekávané změny na obrazovce.

B. Vlastnosti a metody DOM

Co jsou vlastnosti prvků DOM?

Když je webová stránka načtena do prohlížeče, prohlížeč vytvoří Document Object Model (DOM), který reprezentuje strukturu stránky jako strom objektů. Každý HTML prvek se stává DOM objektem, který má vlastnosti a metody dostupné přes JavaScript.

- **Vlastnosti prvků DOM:**
 - `innerText` , `textContent` , `innerHTML` : Přístup nebo změna textového a HTML obsahu prvku.
 - `value` : U formulářových prvků jako `input`, získání nebo nastavení uživatelem zadané hodnoty.
 - `style` : Přístup k in-line CSS vlastnostem.
 - `classList` : Přidání, odebrání nebo přepínání CSS tříd bez přímé manipulace s atributem `class` jako textem.
- **Běžné metody DOM:**
 - **Metody výběru:**
 - `document.getElementById('id')` : Vybere prvek podle ID.
 - `document.querySelector('selector')` : Vybere první prvek odpovídající CSS selektoru.

- `document.querySelectorAll('selector')` : Vybere všechny prvky odpovídající CSS selektoru a vrátí `NodeList`.
- **Vlastní atributy:**
 - `data-*` : Vlastní atributy umožňují ukládat další informace o prvku bez ovlivnění vzhledu či základního chování. Jsou užitečné v JavaScriptu pro přiřazení kontextu či konfigurace.
 - Data atributy jsou velmi užitečné v testovací automatizaci. Nastavením atributů jako `data-cy`, `data-test` nebo `data-testid` vytvoříte stabilní selektory odolné proti změnám UI. Tyto atributy nejsou viditelné pro uživatele a můžete je měnit bez vlivu na vzhled. Testovací skripty pak spolehlivě vybírají prvky pomocí selektorů typu `cy.get('[data-test="login-button"]')` například v Cypressu.
- **Metody manipulace:**
 - `element.appendChild(node)` : Přidá nový podřízený uzel k prvku.
 - `element.removeChild(node)` : Odebere podřízený uzel.
 - `element.setAttribute('name', 'value')` : Nastaví nebo změní atribut prvku.
 - `element.removeAttribute('name')` : Odebere atribut.
- **Zpracování událostí (stručně):**
 - `element.addEventListener('event', callback)` : Připojení posluchače událostí k prvku DOM.
- **Použití řídicích struktur a datových struktur:**
 - Smyčky a podmínky pro dynamickou aktualizaci prvků v DOM (například průchod polem pro vytvoření seznamu položek).
 - Objekt pro ukládání selektorů nebo referencí DOM pro opětovné použití.
 - Data ve formátu JSON lze získat a použít pro naplnění prvků, čímž se DOM aktualizuje za běhu.

1. `innerText` / `textContent` :

- **Účel:** Přístup nebo změna textového obsahu prvku.
- **Příklad:**

```
const heading = document.getElementById('main-title');
console.log(heading.innerText); // Čte zobrazený text nadpisu
heading.innerText = "Welcome to the Dashboard!";
```

- **Rozdíl:** `innerText` respektuje styl a skryté prvky, `textContent` zobrazí všechnen text včetně skrytých prvků a zalomení řádků.

2. `innerHTML` :

- **Účel:** Přístup nebo změna HTML značek uvnitř prvku.
- **Příklad:**

```
const container = document.querySelector('#content');
container.innerHTML = "<p>New paragraph!</p>"; // Vloží HTML kód
```

- **Varování:** Použití `innerHTML` s nedůvěryhodnými daty může vystavit stránku bezpečnostním rizikům, jako je XSS.

3. `value` :

- **Účel:** U formulářových prvků (`inputs`, `textarea`) získává nebo nastavuje hodnotu zadanou uživatelem.
- **Příklad:**

```
const input = document.getElementById('username');
input.value = "testUser";
```

4. `classList` :

- **Účel:** Správa CSS tříd prvku bez ruční úpravy atributu `class` jako textu.
- **Příklad:**

```
const button = document.querySelector('.btn');
button.classList.add('active');
button.classList.remove('disabled');
button.classList.toggle('hidden');
```

5. `style` :

- **Účel:** Přístup nebo změna in-line CSS stylů prvku.
- **Příklad:**

```
const box = document.querySelector('.box');
box.style.backgroundColor = "blue";
box.style.color = "white";
```

- **Poznámka:** Obecně je lepší měnit třídy a používat externí CSS než nastavovat styly přímo v in-line, kvůli přehlednosti a údržbě.

Přístup k prvkům DOM

Výběr prvků:

```
<!-- index.html -->
<div id="container">
  <h1 class="title">Hello, World!</h1>
  <button id="clickMeBtn">Click Me</button>
</div>

// JavaScript
const container = document.getElementById("container");
const title = document.querySelector(".title");
const button = document.querySelector("#clickMeBtn");
```

Nejlepší postupy:

- Používat `document.getElementById()` při výběru podle ID kvůli výkonu.
- Používat `document.querySelector()` a `document.querySelectorAll()` pro složitější nebo flexibilní selekci.
- Udržovat ID a názvy tříd popisné pro lepší srozumitelnost selektorů.

Manipulace s prvky DOM

Změna obsahu:

```
title.innerText = "Welcome to the Test Page!";
```

Úprava atributů:

```
button.setAttribute("disabled", "true");
console.log(button.getAttribute("id")); // "clickMeBtn"
```

Úprava tříd:

```
title.classList.add("highlight");
title.classList.remove("old-class");
```

Nejlepší postupy:

- Minimalizovat přímo manipulaci s DOM cachováním odkazů na prvky.
- Pro změny stylů používat třídy a CSS spíše než in-line styly.
- Hromadit a seskupovat operace s DOM kvůli snížení výkonové náročnosti.

Zpracování událostí

Přidání posluchače událostí:

```
button.addEventListener("click", function () {  
    console.log("Button was clicked!");  
    container.innerHTML += "<p>Button Clicked!</p>";  
});
```

Nejlepší postupy:

- Používat nenápadné zpracování událostí (tj. `addEventListener`) místo in-line atributů jako `onclick` .
- Pojmenovávat funkce pro zpracování událostí popisně:

```
function handleClick(event) {  
    console.log("Button clicked:", event.target);  
}
```

```
button.addEventListener("click", handleClick);
```

- Odebírat posluchače událostí, když již nejsou potřeba, aby nedocházelo k únikům paměti:

```
button.removeEventListener("click", handleClick);
```

B. Globální objekt `window`

Definice:

V internetových prohlížečích je objekt `window` globálním objektem, který reprezentuje okno prohlížeče obsahující vaši webovou stránku. Všechny globální proměnné, funkce a objekty se stávají vlastnostmi objektu `window` .

- **Co je objekt `window` ?**
 - Objekt `window` je globální objekt v prostředí prohlížeče.
 - Reprezentuje okno prohlížeče a nabízí metody a vlastnosti pro jeho ovládání.
- **Klíčové vlastnosti a metody objektu `window` :**
 - **Vlastnosti:**
 - `window.document` : Odkazuje na DOM aktuální stránky.
 - `window.location` : Informace o aktuální URL a navigaci.
 - `window.history` : Přístup k historii relace prohlížeče.

- `window.localStorage` a `window.sessionStorage` : Ukládání a načítání dat v prohlížeči.
- **Metody:**
 - `window.alert('message')` : Zobrazení výstražného dialogu.
 - `window.confirm('message')` : Zobrazení potvrzovacího dialogu, vrací `true/false`.
 - `window.setTimeout(callback, milliseconds)` : Zpožděné spuštění kódu.
 - `window.setInterval(callback, milliseconds)` : Opakované spuštění kódu.
- **Implicitní globální rozsah:**
 - Proměnné deklarované na nejvyšší úrovni (bez `let` , `const` nebo `var` ve starším kódu) se mohou stát vlastností `window` —tuto praxi je lepší se vyhnout.
 - Pochopení rozsahu proměnných (viz předchozí lekce) pomáhá zabránit znečištění globálního jmenného prostoru.

Klíčové body o `window` :

- **Globální rozsah:** Proměnné deklarované globálně (bez `let` , `const` , nebo `var` v starém kódu) se mohou stát vlastnostmi objektu `window` . Moderní best practices toto nedoporučují.
- **Přístup k dokumentu a API prohlížeče:**
 - `window.document` poskytuje DOM aktuální stránky.
 - `window.location` dává info a metody pro aktuální adresu URL.
 - `window.history` umožňuje pohybovat se v historii relace prohlížeče.
 - `window.alert()` , `window.confirm()` , `window.prompt()` zobrazují dialogy.
 - `window.setTimeout()` a `window.setInterval()` plánují spuštění kódu.

Příklad:

```
console.log(window.location.href); // Vypíše aktuální URL stránky
window.alert("Welcome to the site!");
```

Doporučená praxe:

Nevazte zbytečná data na objekt `window` . Proměnné uchovávejte v rozsahu funkcí nebo modulů, abyste předešli znečištění globálního prostoru.

D. Nejlepší postupy při práci s DOM a `window`

- **Údržba kódu:**
 - Uchovávejte reference na často používané prvky v proměnných.
 - Používejte popisná jména proměnných pro referenci na DOM prvky a vyhněte se "magickým řetězcům".
- **Výkonnost:**

- Minimalizujte zbytečné manipulace s DOM—hromadná změna nebo techniky pro omezení reflow a repaint.
- Znovu využívejte pole, objekty a data z JSON pro generování či aktualizaci DOM místo pevně zapsaných prvků.
- **Bezpečnost a validace dat:**
 - Nepoužívejte `innerHTML` s nedůvěryhodnými daty (riziko XSS).
 - Validujte JSON a vstupy od uživatelů před vložením do DOM.
- **Použití řídicích struktur:**
 - Smyčky a podmínky k dynamickým aktualizacím DOM dle uživatelských akcí nebo dat z API.
 - Pokud například máte pole produktů v JSON, můžete je procházet a generovat prvky DOM pro každý produkt.

E. Praktické příklady

1. Přístup a aktualizace obsahu prvku:

```
const title = document.getElementById('page-title');
title.innerText = "Welcome to the Dashboard!";
```

2. Úprava atributů a tříd prvku:

```
const loginButton = document.querySelector('#login-btn');
loginButton.setAttribute('disabled', 'true');
```

```
const mainHeader = document.querySelector('h1');
mainHeader.classList.add('highlighted');
mainHeader.classList.remove('old-class');
```

3. Dynamické vytváření a přidávání prvků:

```
const userList = document.getElementById('user-list');
const users = [
  { name: "Alice", role: "admin" },
  { name: "Bob", role: "user" },
  { name: "Charlie", role: "moderator" }
];

users.forEach(user => {
  const li = document.createElement('li');
  li.innerText = `${user.name} (${user.role})`;
  userList.appendChild(li);
});
```

4. Získání JSON dat a aktualizace DOM:

Co je `fetch()` ?

`fetch()` je vestavěná funkce JavaScriptu (součást Fetch API), která umožňuje provádět síťové požadavky, například získání dat z URL. Vrací Promise, což usnadňuje práci s asynchronními operacemi bez blokování hlavního vlákna.

```
fetch('users.json')
  .then(response => response.json())
  .then(data => {
    const userContainer = document.querySelector('#user-container');
    data.forEach(user => {
      const div = document.createElement('div');
      div.innerText = `${user.name} - ${user.email}`;
      userContainer.appendChild(div);
    });
  });
```

5. `setTimeout`

Co je `setTimeout()` ?

`setTimeout()` je funkce poskytovaná prohlížečem (přes `window`), která umožňuje provést zadanou funkci po určité prodlevě (v milisekundách).

Syntaxe:

```
window.setTimeout(() => {
  console.log("This message appears after 2 seconds");
}, 2000);
```

Parametry:

1. **Callback funkce:** Funkce, kterou chcete spustit po prodlevě.
2. **Prodleva (milisekundy):** Doba čekání, než se zavolá callback. 1000 ms = 1 sekunda.

Chování:

- Callback se provede až po uplynutí zadané doby.
- Kód neblokuje; callback naplánuje a pokračuje v běhu zbytku skriptu. Když čas uplyne, callback se vykoná.

Použití v testovací automatizaci:

- **Simulace čekací doby uživatele:** Například chcete otestovat chování UI po prodlevě (například zmizení spinneru po několika vteřinách).
- **Plánování asynchronních kontrol:** Spuštění kódu po prodlevě pro ověření, zda byla splněna určitá podmínka v DOM.

Zastavení timeoutu:

- Výsledek `setTimeout()` si můžete uložit do proměnné a kdykoli zrušit pomocí `clearTimeout()`, pokud by bylo potřeba.

```
const timeoutId = setTimeout(() => {
  console.log("Will this message appear?");
}, 5000);
```

```
// Zrušení timeoutu dřív, než se spustí
clearTimeout(timeoutId);
```

Důležitost JSON v testovací automatizaci:

JSON (JavaScript Object Notation) je lehce čitelný datový formát pro lidi i stroje. Je nezávislý na jazyce a široce používaný pro komunikaci mezi klienty a servery. V testovací automatizaci:

1. Fixtures a testovací data:

Testovací vstupy a očekávané výstupy můžete ukládat do JSON souborů. Tím oddělujete data od testovací logiky, což zjednodušuje úpravu, údržbu a opakované použití testovacích scénářů.

2. API testování:

Při testování aplikací komunikujících s backendem bývají odpovědi často ve formátu JSON. Parsování JSON odpovědí a kontrola, zda vrácená data odpovídají očekávání, je snadná.

3. Mockování odpovědí:

V automatizovaných testech můžete poskytovat předem definovaná JSON data jako mock odpovědi. Tím zajistíte deterministické testy bez závislosti na externích službách.

Parsování a práce s JSON:

• Metoda `response.json()` :

Po zavolání `fetch()` často převedete syrovou odpověď na JSON:

```
fetch('users.json')
  .then(response => response.json()) // parsuje JSON z odpovědi
  .then(data => {
    console.log(data);
    // data je nyní JavaScript objekt/pole, se kterým lze dále pracovat
  });
```

Co znamená `then` ve funkci `fetch()` ?

Vysvětlení:

`then()` je metoda dostupná na Promise objektech. Při volání `fetch()` se vrátí Promise. Metoda `then()` určuje, co se má stát, až bude Promise vyřešeno (tj. operace úspěšně dokončena).

- **První `.then()`** : Obvykle pro zpracování surové odpovědi z `fetch()` a převod na použitelný formát (například JSON).
- **Druhé `.then()`** : Jakmile máte parsovaná data, můžete navázat další `then()` pro jejich použití (například aktualizace DOM, kontrola výsledku atd.).

Příklad:

```
fetch('users.json')
  .then(response => response.json())    // Převede tělo odpovědi na JSON
  .then(data => {
    console.log(data);                 // Zde lze s daty dále pracovat
  })
  .catch(error => console.error(error));
```

Klíčové body:

- `then()` slouží k řetězení asynchronních operací. Každý `then()` vrátí nový Promise, takže lze v řetězení pokračovat.
- `catch()` je použit k ošetření chyb, pokud některá část řetězce selže.

5. Použití objektu `window` :

```
console.log(window.location.href); // Aktuální URL stránky
window.setTimeout(() => {
  alert("Time's up!");
}, 2000);
```

F. Propojení na předchozí lekce

- **Pole a objekty:**
 - Používejte pole pro uložení seznamu dat a vytvářejte prvky DOM dynamicky pomocí smyček.
 - Objekty použijte k uchování selektorů nebo referencí pro jednodušší práci s DOM.
- **JSON a interakce s API:**
 - Načítejte JSON data a používejte je pro doplnění prvků na stránce.
- **Řídící struktury:**

- Pokud/else podmínky, smyčky a logické podmínky určují, jak a kdy aktualizovat DOM, zpracovávat vstupy uživatele, nebo měnit vlastnosti window .
- **Kombinace konceptů:**
 - Vše, co jste se naučili—proměnné, funkce, pole, objekty, JSON, manipulace s DOM, zpracování událostí—lze nyní spojit pro tvorbu interaktivních a dynamických webových aplikací.

2. Praktické aktivity: Cvičení a návrhy webových funkcí

A. Interakce s DOM v testech – cvičení

- **Cvičení:**
 - Vytvořte HTML stránku s různými prvky jako tlačítka, pole pro zadání textu a kontejnery.
 - Napište testy v Cypress, které:
 - Vyberou a interagují s prvky pomocí selektorů.
 - Ověří přítomnost a obsah prvků.
 - Simulují uživatelské interakce jako kliknutí a odeslání formuláře.
 - **Příklad:**

```
<!-- index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>DOM Interaction Test</title>
</head>
<body>
  <h1 id="title">Welcome to the Test Page</h1>
  <button id="changeTitle">Change Title</button>
  <input type="text" id="username" placeholder="Enter username" />
  <button id="submitForm">Submit</button>
  <div id="output"></div>

  <script>
    document.getElementById('changeTitle').addEventListener('click', () => {
      document.getElementById('title').innerText = 'Title Changed!';
    });

    document.getElementById('submitForm').addEventListener('click', () => {
      const username = document.getElementById('username').value;
      document.getElementById('output').innerText = `Hello, ${username}!`;
    });
  </script>
</body>
</html>
```

```
// Cypress Test
describe('DOM Interaction Test', () => {
  beforeEach(() => {
    cy.visit('/index.html');
  });

  it('Changes the title when button is clicked', () => {
    cy.get('#changeTitle').click();
    cy.get('#title').should('have.text', 'Title Changed!');
  });

  it('Submits the form and displays greeting', () => {
    cy.get('#username').type('TestUser');
    cy.get('#submitForm').click();
    cy.get('#output').should('have.text', 'Hello, TestUser!');
  });
});
```

- **Návrh funkčnosti:**

- Navrhnete dynamickou webovou stránku, kde uživatelé mohou:
 - Změnit titulek stránky kliknutím na tlačítko.
 - Odeslat formulář s uživatelským jménem a zobrazit personalizovaný pozdrav.
- Vytvořte Cypress testy pro ověření těchto interakcí.

B. Cvičení na aktualizaci obsahu DOM

- **Cvičení:**

- Vytvořte jednoduchou stránku s titulkem, odstavcem a tlačítkem.
- Napište JavaScript, který:
 - Změní text titulku pomocí `innerText` při načtení stránky.
 - Aktualizuje obsah odstavce po kliknutí na tlačítko.
- **Návrh webové funkčnosti:**
 - “Vítací” stránka, která zobrazuje obecnou zprávu a po kliknutí na tlačítko zobrazí personalizovaný pozdrav.

C. Cvičení na dynamické vytváření seznamu z JSON

- **Cvičení:**

- Vytvořte JSON soubor (`products.json`) s polem produktů (každý má `name` , `price` , `category`).
- Pomocí `fetch()` načtěte JSON data.

- Dynamicky vytvořte na webu seznam produktů, kde se zobrazí název a cena každého produktu.
- **Návrh webové funkčnosti:**
 - Stránka s výpisem produktů, načítajících se z datového souboru, zobrazených v tabulce nebo seznamu, s možností aktualizace dat.

D. Použití metod `window` – cvičení

- **Cvičení:**
 - Po 2 vteřinách zobrazit upozornění pomocí `window.setTimeout()` .
 - Zalogovat aktuální URL stránky pomocí `window.location.href` .
 - Implementujte tlačítko „Obnovit stránku“, které použije `window.location.reload()` pro aktualizaci.
 - **Návrh webové funkčnosti:**
 - Stránka s časovaným oznámením a tlačítkem pro reload, demonstrující manipulaci s `window` .

3. Možné otázky studentů

1. Jaký je rozdíl mezi `document.getElementById()` a `document.querySelector()` ?

• Odpověď:

`document.getElementById()` vybírá prvek podle jeho unikátního ID a je obecně rychlejší.
`document.querySelector()` umožňuje vybírat prvky pomocí jakéhokoliv CSS selektoru a je flexibilnější.

2. Jak mohu upravit obsah prvku DOM pomocí JavaScriptu?

• Odpověď:

Můžete použít vlastnosti jako `innerText` , `innerHTML` nebo `textContent` . Například:

```
document.getElementById('title').innerText = 'New Title';
```

3. Kdy použít `innerText` vs. `innerHTML` ?

Odpověď:

- `innerText` nastavuje nebo získává pouze čitelný text uvnitř prvku, ignoruje HTML značky.
- `innerHTML` umožňuje přidávat nebo upravovat HTML přímo. S `innerHTML` opatrně, protože lze snadno způsobit XSS.

4. Jak zamezit znečištění globální proměnné pomocí `window` ?

Odpověď:

- Vždy deklarujte proměnné pomocí `let` nebo `const` uvnitř funkcí nebo bloků.

- Používejte moduly či closure pro udržení proměnných v lokálním rozsahu.
- Nespoléhejte na připojování dat k objektu `window`, místo toho je uchovávejte ve vlastních objektech či modulech.

5. Jaký je rozdíl mezi `document.getElementById()` a `document.querySelector()` ?

Odpověď:

- `getElementById()` vybírá prvek podle unikátního ID a vrátí jeden prvek.
- `querySelector()` používá CSS selektory, může vybrat jakýkoliv prvek odpovídající selektoru a vrátí první shodu.
- `querySelector()` je flexibilnější, ale `getElementById()` je rychlejší pro hledání jediného ID.

6. Mohu iterovat kolekci prvků DOM jako pole?

Odpověď:

- `querySelectorAll()` vrátí `NodeList`, který lze procházet pomocí `forEach()`.
- `NodeList` nebo `HTMLCollection` lze pomocí `Array.from()` převést na pole a použít metody pole.

4. Doplnkové materiály: Doporučení

A. Oficiální dokumentace a průvodci:

- [MDN Web Docs - Úvod do DOM](#)
- [MDN Web Docs - Objekt Window](#)
- [MDN Web Docs - Manipulace s DOM](#)

B. Tutoriály a články:

- **W3Schools DOM Tutorial:** https://www.w3schools.com/js/js_htmlDOM.asp
- **JavaScript.info DOM Tutorial:** <https://javascript.info/document>

C. Interaktivní vzdělávací platformy:

- **FreeCodeCamp:** Cvičení na manipulaci s DOM a zpracování událostí.
- **Codecademy:** Interaktivní lekce o práci s DOM.

D. Video tutoriály:

- **Traversy Media na YouTube:** Video zaměřená na základy manipulace s DOM.
- **The Net Ninja:** Tutoriály DOM, vysvětlení vlastností, metod, i objektu `window`.

E. Tréninkové platformy:

- Vytvářejte malé demo stránky pro zkoušení různých metod DOM.
- Používejte JSON soubory jako zdroj dat a dynamicky vykreslujte UI prvky na základě získaných dat.

F. Komunita a podpora:

- **Stack Overflow** a **Reddit** (`r/javascript`): Výborné zdroje pro konkrétní dotazy týkající se DOM.
- **Discord komunity**: Přidejte se do kanálů zaměřených na JavaScript či front-end a diskutujte o výzvách v DOM.

5. Návrh rozložení lekce na 3 hodiny

Hodina 1: Vlastnosti a metody DOM (60 minut)

- **Rekapitulace pojmu DOM (10 minut):**
 - Připomenutí struktury DOM a způsobu, jak JavaScript vnímá stránku.
- **Vlastnosti a metody DOM (40 minut):**
 - `innerText` , `innerHTML` , `textContent` .
 - `classList` a `style` .
 - Tvorba, úprava a odstraňování prvků.
- **Krátké Q&A (10 minut):**
 - Okamžitá reakce na dotazy.

Hodina 2: Globální objekt Window a integrace (60 minut)

- **Zkoumání `window` (20 minut):**
 - `window.document` , `window.location` , `window.history` .
 - Časované akce (`setTimeout` , `setInterval`).
- **Praktická integrace s daty (20 minut):**
 - Stažení JSON dat a jejich použití v elementech DOM.
 - Smyčky a podmínky pro rozhodování, co ukázat.
- **Krátké Q&A (10 minut):**
 - Odpovědi, upřesnění.
- **Pauza (10 minut):**
 - Krátký oddech.

Hodina 3: Praktická cvičení a osvojení Best Practices (60 minut)

- **Praktická cvičení (40 minut):**

- Aktualizace textu a atributů.
- Tvorba seznamů z polí nebo JSON.
- Použití metod `window` k zobrazení alertů nebo obnovení stránky.

- **Shrnutí a Q&A (20 minut):**

- Rekapitulace klíčových konceptů a postupů.
- Povzbuzení k experimentování a zkoušení pokročilejších manipulací s DOM.

6. Další doporučení

A. Interaktivní ukázky:

- Živé kódování:
 - Načtení JSON souboru a zobrazení dat na stránce.
 - Update tříd elementů po kliknutí na tlačítko.
 - Zobrazení časovaného alertu pomocí `setTimeout()` .

B. Zapojení vizuálních prvků:

- Použijte diagramy pro znázornění stromu DOM.
- Flowcharty pro vizualizaci toku dat od JSON k prvkům DOM.
- Zvýrazněte, jak `window` stojí na nejvyšší úrovni prostředí prohlížeče.

C. Podpora participace:

- Vyžádejte, aby studenti našli a zalogovali `window.location.href` .
- Nechte studenty vytvářet a odstraňovat prvky za běhu.
- Ať si zkusí výběr prvků pomocí různých metod a vlastností.

D. Jasně instrukce:

- Kroková zadání, například „změňte text nadpisu a přidejte nový odstavec pod něj“.
- Tipy na řešení běžných chyb (např. špatný ID selektor, překlep v selektoru).

E. Vstřícné prostředí:

- Povzbuzujte studenty ke sdílení kódu.
- Nabídněte zpětnou vazbu a pomoc s jejich řešením.